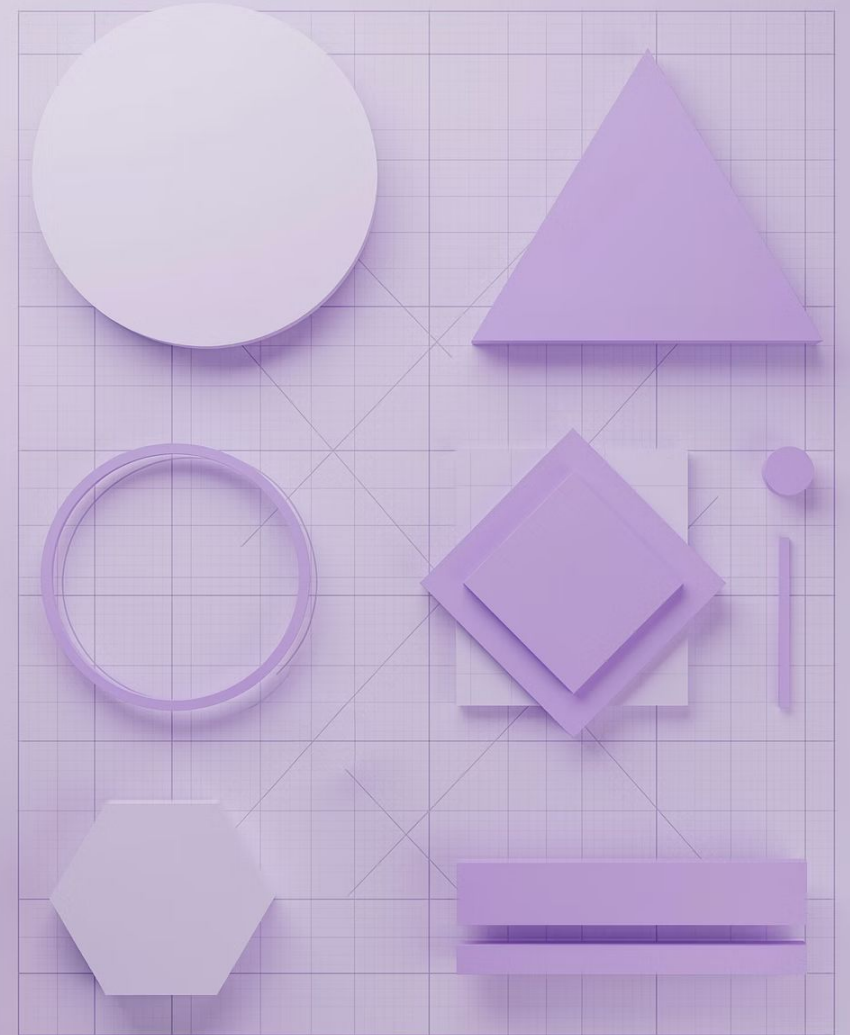


PROGRAMACIÓN ORIENTADA A OBJETOS

Atributos derivados

El dato que no se guarda, se calcula

Una clase más limpia, sin inconsistencias: así se diseña con **@property**.



¿Qué es un atributo derivado?

Un **atributo derivado** no vive guardado en el objeto: **se calcula a partir de otros datos** cada vez que se pide. No ocupa espacio propio en el estado del objeto, porque no es un dato nuevo: es una **consecuencia** de datos que ya existen.

La diferencia con un atributo normal es sutil pero crítica. Un atributo guardado se almacena en memoria y puede desincronizarse si los datos de los que depende cambian. Un atributo derivado siempre refleja el estado real del objeto, porque se recalcula en el momento.

El test de reconocimiento

"¿Este dato cambia solo si cambian los datos de los que depende, sin que nadie lo actualice a mano?"

- **Sí** → es derivado → usar **@property**
- **No** → es un atributo que se guarda



Atributo guardado

Se asigna con **self.x = valor**. Vive en el objeto. Hay que actualizarlo a mano si cambia algo.



Atributo derivado

Se define con **@property**. Se calcula cada vez que se accede. Nunca se desincroniza.



La señal en el enunciado

Palabras clave: "**se calcula**", "**depende de**", "**surge de**", "**es igual a**".

Polígonos: lados y perímetro

Este ejemplo muestra cómo varios datos de una misma clase son derivados, incluyendo uno que depende de datos aún más básicos.



Enunciado

"El perímetro y la cantidad de lados de un polígono **no se cargan a mano**: se calculan a partir de los lados que efectivamente tiene."



nro_lados

No se guarda: es la **cantidad** de elementos en la lista de lados.
Se deriva contando los lados existentes.



perimetro

No se guarda: es la **suma** de las longitudes de todos los lados.
Se recalcula cada vez que se pide.



longitud de cada lado

¡También es derivada! Se calcula a partir de sus **dos puntos extremos**. Un derivado puede depender de otro derivado.

Control de gasto público: Ciudad

Enunciado: "Una ciudad recauda varios impuestos y tiene un gasto de mantenimiento. Está **controlada** si supera cierta población, y está en **déficit** si el gasto supera lo recaudado."

Ninguno de esos tres datos se guarda en el objeto. Los tres **se derivan** de datos más simples que sí se almacenan.

1 total_recaudado

Suma de todos los impuestos que recibe la ciudad. Se calcula a partir de la lista de impuestos.

2 controlada

Verdadero si la población supera el umbral definido. Depende del atributo guardado **poblacion**.

3 en_deficit

Verdadero si el gasto de mantenimiento supera el total recaudado. Depende de dos valores que sí se guardan.

¿Qué se guarda y qué se deriva?

- **Se guarda:** lista de impuestos, gasto_mantenimiento, poblacion, umbral_control
- **Se deriva:** total_recaudado, controlada, en_deficit

Los datos guardados son los **hechos brutos**. Los derivados son las **conclusiones** que el sistema puede sacar a partir de esos hechos.

Derivados que dependen de derivados — y filtros que cambian todo

A veces el enunciado esconde una complejidad extra: hay que **filtrar** antes de calcular, y el resultado de ese cálculo puede ser, a su vez, la base de otro atributo derivado.

El caso: Provincia en riesgo

"Una provincia está **en riesgo** si la mitad de sus ciudades controladas está en déficit."

La palabra "**controladas**" no es decorativa: **filtra el universo** antes de calcular el porcentaje. No se divide entre todas las ciudades, sino solo entre las que cumplen una condición previa.

Esto significa que **en_riesgo** depende de **controlada** y de **en_deficit** — ambos, a su vez, son @property de Ciudad.

¿Cómo leer el enunciado?

→ Identificar la condición de filtro

"sus ciudades **controladas**" → no son todas las ciudades, solo las que cumplen una propiedad derivada.

→ Identificar qué se mide sobre ese filtro

"la **mitad** está en déficit" → se compara una fracción sobre el subconjunto filtrado.

→ Reconocer que no es un atributo nuevo

La regla se aplica sobre derivados ya existentes. No hay que inventar atributos extras: hay que **componerlos**.

LA TRAMPA A RECORDAR

Guardar un derivado es un bug esperando pasar

Si el enunciado usa estas palabras, es una **@property**, no un atributo para guardar:

"se calcula"

El valor emerge de una operación sobre otros datos del objeto.

"depende de"

El valor es consecuencia directa de uno o más atributos ya existentes.

"surge de"

El dato aparece naturalmente al combinar o evaluar otros datos del objeto.

Si guardás el resultado en lugar de derivarlo, el dato guardado **se desincroniza** en cuanto cambia algo de lo que depende. El objeto empieza a mentir sobre su propio estado.



¿Guardado o derivado? El cuadro comparativo

Ante cada dato del modelo, hacé la pregunta del test. Si la respuesta es sí, va como **@property**.

Atributo guardado

Se asigna en **__init__**. Vive en el objeto. Es un hecho primitivo que no se puede calcular a partir de otros datos internos.

habitantes

Dato de entrada de Ciudad. No se puede calcular internamente.

gasto_mantenimiento

Costo real de la ciudad. Es un valor externo que se carga.

lados (lista cargada)

Los lados del polígono se reciben y almacenan tal como vienen.

impuestos (lista)

Lista de valores recaudados. Son hechos brutos, no calculados.

Atributo derivado / @property

Se define con **@property**. Se recalcula cada vez que se accede. Siempre es consistente.

total_recaudado

Suma de la lista de impuestos de Ciudad.

controlada

Depende de habitantes vs. umbral de Ciudad.

en_deficit

Depende de gasto_mantenimiento vs. total_recaudado.

perimetro

Suma de las longitudes de los lados del polígono.

nro_lados

Cantidad de elementos en la lista de lados.